

```
In [113... #import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sqlite3
```

```
In [114... # import database / data
```

```
In [115... conn = sqlite3.connect('customer_churn.db')

sql_query = """
    SELECT name
    FROM sqlite_master
    WHERE type='table'
    """

tables = pd.read_sql(sql_query, conn)

#create DataFrame for each table
for table_name in tables['name']:
    df = pd.read_sql(f"SELECT * FROM {table_name}", conn)
    globals()[f"df_{table_name}"] = df
    print(f"Created dataframe: df_{table_name}")

conn.close()
```

Created dataframe: df_db_customer
Created dataframe: df_db_subscription
Created dataframe: df_db_support

```
In [116... # Print table names and column names
conn = sqlite3.connect('customer_churn.db')

for table_name in tables['name']:
    print(f"\nTable Name: {table_name}")

    # Get column information
    columns_query = f"PRAGMA table_info({table_name});"
    columns = pd.read_sql(columns_query, conn)
    print("Columns: ")
    print(columns['name'].tolist())

# Close connection
conn.close()
```

Table Name: db_customer
Columns:
['customerid', 'name', 'country', 'state', 'gender', 'dob', 'interests', 'pincode']

Table Name: db_subscription
Columns:
['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score']

Table Name: db_support
Columns:
['customerid', 'complaint_date', 'escalations', 'csat_score', 'col_1', 'comment']

```
In [117... # Data cleaning
```

```
In [118... df_db_customer.head()
```

Out[118...<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th><th>customerid</th> <th>name</th> <th>country</th> <th>state</th> <th>gender</th><th>dob</th> <th>interests</th> <th>pincode</th> </thead> <tbody> <th>0</th> <th>1</th><th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00	travel	None
0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00	NaN	None
0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00	movie	None
0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00	NaN	None
0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00	drama	None

```
In [119... df_db_customer.tail()
```

Out[119...<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th><th>customerid</th> <th>name</th> <th>country</th> <th>state</th> <th>gender</th><th>dob</th> <th>interests</th> <th>pincode</th> </thead> <tbody> <th>16</th> <th>17</th><th>18</th> <th>19</th> <th>20</th> </tbody> <tbody></tbody>

0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00	NaN	None
0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00	NaN	None
0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00	NaN	None
0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00	NaN	None
0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00	NaN	None

```
In [120... df_db_customer.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
---  -
0   customerid  21 non-null    str
1   name        21 non-null    str
2   country     18 non-null    str
3   state       21 non-null    str
4   gender      21 non-null    str
5   dob         21 non-null    str
6   interests   4 non-null     str
7   pincode     0 non-null     object
dtypes: object(1), str(7)
memory usage: 1.4+ KB
```

```
In [121... # a. rename col - name
# b. drop columns - intrest and pincode
# c. change datatype - dob
# d. data standardization - gender
# e. fix missing value - country
```

```
In [122... # a. rename col - name
df_db_customer.rename(columns = {'name': 'customer_name'}, inplace= True)
```

```
In [123... #b. drop columns - interest and pincode

#df_db_customer.drop(df_db_customer.columns[-2:], axis=1)
#df_db_customer.drop(df_db_customer.columns[6:], axis=1)

df_db_customer.drop(columns=['interests', 'pincode'], inplace= True )
```

```
In [124... df_db_customer.info()

<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   customerid      21 non-null    str
1   customer_name   21 non-null    str
2   country         18 non-null    str
3   state           21 non-null    str
4   gender          21 non-null    str
5   dob             21 non-null    str
dtypes: str(6)
memory usage: 1.1 KB
```

```
In [125... # c. change data type - dob

df_db_customer['dob'] = pd.to_datetime(df_db_customer['dob'])
```

```
In [126... #d. data standardization - gender

#df_db_customer['gender'].unique()
df_db_customer['gender'] = df_db_customer['gender'].replace({'Men': 'Male', 'Women' : 'Female'}
```

```
In [127... df_db_customer['gender'].unique()
```

```
Out[127... <StringArray>
['Male', 'Female']
Length: 2, dtype: str
```

```
In [128... #e. fix missing values - country

df_db_customer[df_db_customer['country'].isna()]
```

```
Out[128... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th {
vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th>
<th>customerid</th> <th>customer_name</th> <th>country</th> <th>state</th> <th>gender</th>
<th>dob</th> </thead> <tbody> <tr><td>5</td> <td>8</td> <td>12</td> </tbody> <tbody></tbody>
0013-MHZWF  durga  NaN      Delhi  Female  1988-12-10
0015-UOCOJ  maya   NaN      Kathmandu  Female  1985-07-07
0018-NYROU  chitra  NaN      Telangana  Female  2004-12-01
```

```
In [129... df_db_customer[['country', 'state']]
```

Out[129...

```
<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th {
vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th>
<th>country</th> <th>state</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th>
<th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th>
<th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> <th>16</th> <th>17</th>
<th>18</th> <th>19</th> <th>20</th> </tbody> <tbody></tbody>
```

India	Maharashtra
-------	-------------

India	Karnataka
-------	-----------

India	Delhi
-------	-------

India	Nagaland
-------	----------

India	Delhi
-------	-------

NaN	Delhi
-----	-------

India	Meghalaya
-------	-----------

India	Rajasthan
-------	-----------

NaN	Kathmandu
-----	-----------

Nepal	Kathmandu
-------	-----------

India	Maharashtra
-------	-------------

India	Karnataka
-------	-----------

NaN	Telangana
-----	-----------

India	Meghalaya
-------	-----------

India	Uttar Pradesh
-------	---------------

India	Delhi
-------	-------

India	Meghalaya
-------	-----------

India	Rajasthan
-------	-----------

India	Telangana
-------	-----------

India	Uttar Pradesh
-------	---------------

India	Maharashtra
-------	-------------

In [130...

```
# country and state - unique value pair

state_country_mapping = df_db_customer.dropna(subset=['country']).set_index('state')['country']

df_db_customer['country'] = df_db_customer['country'].fillna(df_db_customer['state'].map(state_country_mapping))
```

In [131...

```
df_db_customer[df_db_customer['country'].isna()]
```

Out[131...

```
<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th {
vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th>
<th>customerid</th> <th>customer_name</th> <th>country</th> <th>state</th> <th>gender</th>
<th>dob</th> </thead> <tbody> </tbody> <tbody></tbody>
```

In [132...

```
df_db_customer[['country', 'state']]
```

Out[132...

```
<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th {
vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th>
<th>country</th> <th>state</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th>
<th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th>
<th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> <th>16</th> <th>17</th>
<th>18</th> <th>19</th> <th>20</th> </tbody> <tbody></tbody>
```

India	Maharashtra
-------	-------------

India	Karnataka
-------	-----------

India	Delhi
-------	-------

India	Nagaland
-------	----------

India	Delhi
-------	-------

India	Delhi
-------	-------

India	Meghalaya
-------	-----------

India	Rajasthan
-------	-----------

Nepal	Kathmandu
-------	-----------

Nepal	Kathmandu
-------	-----------

India	Maharashtra
-------	-------------

India	Karnataka
-------	-----------

India	Telangana
-------	-----------

India	Meghalaya
-------	-----------

India	Uttar Pradesh
-------	---------------

India	Delhi
-------	-------

India	Meghalaya
-------	-----------

India	Rajasthan
-------	-----------

India	Telangana
-------	-----------

India	Uttar Pradesh
-------	---------------

India	Maharashtra
-------	-------------

In [133...

```
df_db_subscription.head()
```

Out[133...

```
<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th {
vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th>
<th>customerid</th> <th>subscription_start_date</th> <th>subscription_type</th>
<th>renewal_date</th> <th>plan_type</th> <th>contract_type</th> <th>cancellation_date</th>
<th>cancellation_reason</th> <th>monthly_charges</th> <th>cltv</th> <th>churn_score</th>
</thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody>
</tbody>
```

0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaN	NaN	13.99	627	12
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	91
0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaN	NaN	6.99	210	34
0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	NaN	NaN	22.99	1725	8
0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	Too expensive	13.99	195	88

In [134...

```
df_db_subscription.info()

<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   customerid                           21 non-null    str
1   subscription_start_date               21 non-null    str
2   subscription_type                     21 non-null    str
3   renewal_date                         21 non-null    str
4   plan_type                            21 non-null    str
5   contract_type                        21 non-null    str
6   cancellation_date                    6 non-null     str
7   cancellation_reason                   6 non-null     str
8   monthly_charges                      21 non-null    float64
9   cltv                                 21 non-null    int64
10  churn_score                           21 non-null    int64
dtypes: float64(1), int64(2), str(8)
memory usage: 1.9 KB
```

In [135...

```
date_col = ['subscription_start_date', 'renewal_date', 'cancellation_date']

df_db_subscription[date_col] = df_db_subscription[date_col].apply(pd.to_datetime)
df_db_subscription.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   customerid                           21 non-null     str
1   subscription_start_date               21 non-null     datetime64[us]
2   subscription_type                      21 non-null     str
3   renewal_date                         21 non-null     datetime64[us]
4   plan_type                            21 non-null     str
5   contract_type                        21 non-null     str
6   cancellation_date                    6 non-null      datetime64[us]
7   cancellation_reason                   6 non-null      str
8   monthly_charges                      21 non-null     float64
9   cltv                                 21 non-null     int64
10  churn_score                           21 non-null     int64
dtypes: datetime64[us](3), float64(1), int64(2), str(5)
memory usage: 1.9 KB
```

```
In [136... df_db_support.head()
```

Out[136... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customerid</th> <th>complaint_date</th> <th>escalations</th> <th>csat_score</th> <th>col_1</th> <th>comment</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

0003-MKNFE	2024-08-28 00:00:00	N	60	None		service issue
0003-MKNFE	2024-08-28 00:00:00	Y	10	None		demaned refund
0013-EXCHZ	2024-01-20 00:00:00	Y	20	None		NaN
0013-MHZWF	2025-03-18 00:00:00	N	90	None		guidance to renew
0013-SMEOE	2024-11-01 00:00:00	N	30	None		NaN

```
In [137... df_db_support.tail()
```

Out[137... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customerid</th> <th>complaint_date</th> <th>escalations</th> <th>csat_score</th> <th>col_1</th> <th>comment</th> </thead> <tbody> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> </tbody> <tbody></tbody>

0013-SMEOE	2024-11-01 00:00:00	N	30	None		NaN
0017-IUDMW	2024-04-10 00:00:00	Y	25	None		NaN
0019-EFAEP	2024-09-27 00:00:00	Y	30	None		NaN
0022-TCJCI	2024-09-13 00:00:00	Y	10	None		NaN
0022-TCJCI	2024-09-14 00:00:00	N	90	None		received refund

```
In [138... df_db_support.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   customerid      9 non-null     str
1   complaint_date  9 non-null     str
2   escalations     9 non-null     str
3   csat_score      9 non-null     int64
4   col_1           0 non-null     object
5   comment         4 non-null     str
dtypes: int64(1), object(1), str(4)
memory usage: 564.0+ bytes

```

```
In [139... df_db_support.drop(columns=['col_1', 'comment'], inplace=True)
```

```
In [140... df_db_support['complaint_date'] = pd.to_datetime(df_db_support['complaint_date'])

df_db_support.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   customerid      9 non-null     str
1   complaint_date  9 non-null     datetime64[us]
2   escalations     9 non-null     str
3   csat_score      9 non-null     int64
dtypes: datetime64[us](1), int64(1), str(2)
memory usage: 420.0 bytes

```

```
In [ ]:
```

```
In [141... #3. Feature engineering
```

```
In [142... df_db_subscription['churn_flag'] = np.where(df_db_subscription['cancellation_date'].notna(), 1,
```

```
In [143... df_db_subscription.head()
```


Out[143...<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th><th>customerid</th> <th>subscription_start_date</th> <th>subscription_type</th><th>renewal_date</th> <th>plan_type</th> <th>contract_type</th> <th>cancellation_date</th><th>cancellation_reason</th> <th>monthly_charges</th> <th>cltv</th> <th>churn_score</th><th>churn_flag</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th><th>4</th> </tbody> <tbody></tbody></div><table><tr><td>0002-ORFBO</td><td>2021-03-15</td><td>Refferal</td><td>2025-03-15</td><td>Standard</td><td>Annual</td><td>NaT</td><td>NaN</td><td>13.99</td><td>627</td><td>12</td><td>0</td></tr><tr><td>0003-MKNFE</td><td>2020-08-01</td><td>Paid</td><td>2024-08-01</td><td>Premium</td><td>Annual</td><td>2024-09-10</td><td>Switched to competitor</td><td>12.99</td><td>1150</td><td>91</td><td>1</td></tr><tr><td>0004-TLHLJ</td><td>2022-11-20</td><td>Organic</td><td>2025-11-20</td><td>Basic</td><td>Monthly</td><td>NaT</td><td>NaN</td><td>6.99</td><td>210</td><td>34</td><td>0</td></tr><tr><td>0011-IGKFF</td><td>2019-05-10</td><td>Paid</td><td>2025-05-10</td><td>Premium</td><td>Annual</td><td>NaT</td><td>NaN</td><td>22.99</td><td>1725</td><td>8</td><td>0</td></tr><tr><td>0013-EXCHZ</td><td>2023-01-05</td><td>Refferal</td><td>2024-01-05</td><td>Standard</td><td>Monthly</td><td>2024-02-28</td><td>Too expensive</td><td>13.99</td><td>195</td><td>88</td><td>1</td></tr></table></div><div><div>In [144...<pre>df = (df_db_subscription.
 merge(df_db_customer, on = 'customerid', how = 'left')
 . merge(df_db_support, on = 'customerid', how = 'left'))</pre></div><div>In [145...<pre>df.shape</pre></div><div>Out[145...<pre>(23, 20)</pre></div><div>In [146...<pre>df_db_subscription['customerid'].nunique()</pre></div><div>Out[146...<pre>21</pre></div><div>In [147...<pre>df_db_support</pre></div><div>Out[147...<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th><th>customerid</th> <th>complaint_date</th> <th>escalations</th> <th>csat_score</th> </thead><tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th><th>7</th> <th>8</th> </tbody> <tbody><tr><td>0003-MKNFE</td><td>2024-08-28</td><td>N</td><td>60</td></tr><tr><td>0003-MKNFE</td><td>2024-08-28</td><td>Y</td><td>10</td></tr><tr><td>0013-EXCHZ</td><td>2024-01-20</td><td>Y</td><td>20</td></tr><tr><td>0013-MHZWF</td><td>2025-03-18</td><td>N</td><td>90</td></tr><tr><td>0013-SMEOE</td><td>2024-11-01</td><td>N</td><td>30</td></tr><tr><td>0017-IUDMW</td><td>2024-04-10</td><td>Y</td><td>25</td></tr><tr><td>0019-EFAEP</td><td>2024-09-27</td><td>Y</td><td>30</td></tr><tr><td>0022-TCJCI</td><td>2024-09-13</td><td>Y</td><td>10</td></tr><tr><td>0022-TCJCI</td><td>2024-09-14</td><td>N</td><td>90</td></tr></tbody></table></div></div>

```
In [148... df_db_support['complaint_count'] = df_db_support.groupby('customerid')['customerid'].transform
```

```
In [149... df_db_support = df_db_support.sort_values('complaint_date').drop_duplicates ('customerid', ke
```

```
In [150... # merg df
df = (df_db_subscription.
      merge(df_db_customer, on = 'customerid', how = 'left')
      . merge(df_db_support, on = 'customerid', how = 'left') )
```

```
In [151... df.shape
```

```
Out[151... (21, 21)
```

```
In [152... df.to_csv('exported_churn_data.csv', index=False)
```

```
In [153... # Data Analysis
```

```
In [154... # churn rate
```

```
In [155... df.columns
```

```
Out[155... Index(['customerid', 'subscription_start_date', 'subscription_type',
        'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
        'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
        'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
        'complaint_date', 'escalations', 'csat_score', 'complaint_count'],
      dtype='str')
```

```
In [156... churn_rate = df['churn_flag'].mean()*100
print("Churn Rate = ", round(churn_rate,2), "%")
```

Churn Rate = 28.57 %

```
In [157... #2. Retenion Rate
retention_rate = 100 - churn_rate
print("Rentention Rate = ", round(retention_rate,2), "%" )
```

Rentention Rate = 71.43 %

```
In [158... # 3. Churn by Plan type
churn_by_plan = (df.groupby('plan_type') ['churn_flag'].mean(). mul (100).round(2).reset_index)
print(churn_by_plan)
```

	plan_type	churn_rate_pct
0	Basic	60.00
1	Premium	14.29
2	Standard	22.22

```
In [159... # Average ravenue per user
arpu = df ['monthly_charges'].mean()
print('ARPU = ', round (arpu, 2))
```

ARPU = 18.85

```
In [160... df.columns
```

```
Out[160... Index(['customerid', 'subscription_start_date', 'subscription_type',
        'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
        'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
        'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
        'complaint_date', 'escalations', 'csat_score', 'complaint_count'],
      dtype='str')
```

```
In [161... # 6. Avg Customer Tenure
# count of days users has used our service cancellation date else current date
today = pd.Timestamp.today()

df['tenure_days'] = np.where(
df['cancellation_date'].notna(),
(df['cancellation_date'] - df['subscription_start_date']).dt.days,
(today - df['subscription_start_date']).dt.days)

avg_tenure = df['tenure_days'].mean()
print("Avg Tenure (Days) = ", (avg_tenure),0)
```

Avg Tenure (Days) = 1513.7142857142858 0

```
In [162... #Revenue at risk - revenue lost from churned users
df.loc[df['churn_flag']==1, 'monthly_charges'].sum()
```

Out[162... np.float64(73.94)

```
In [163... # Escalation Rate
escalation_rate = (df['escalations'] == 'Y').mean() * 100
print("Escalation Rate = ", round(escalation_rate, 2), "%")
```

Escalation Rate = 19.05 %

```
In [164... # 9. Avg Complaint Per User
avg_complaints = df['complaint_count'].sum() / df['customerid'].nunique()
print("Avg Compliants Per User = ", round(avg_complaints, 2))
```

Avg Compliants Per User = 0.43

```
In [166... # 10. Correlation Escalation vs Churn
df['escalations'] = np.where(df['escalations'] == 'Y', 1, 0) # encoding string to int type
corr_df = df[['escalations', 'churn_flag']].dropna()

correlation = corr_df['escalations'].corr(df['churn_flag'])

print("Correlation between escalation vs churn is =", round(correlation, 2))
```

Correlation between escalation vs churn is = 0.77

```
In [166... # 11. Churn risk - create a column using existing col
conditions = [
(df['churn_score'] < 50),
(df['churn_score'] >= 50) & (df['churn_score'] < 70),
(df['churn_score'] >= 70)
]
choices = ['low', 'med', 'high']
df['churn_risk'] = np.select(conditions, choices, default='unkown')
```

```
In [167... df[['churn_risk', 'churn_score']].tail()
```

```
Out[167... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th {
vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th>
<th>churn_risk</th> <th>churn_score</th> </thead> <tbody> <th>16</th> <th>17</th>
<th>18</th> <th>19</th> <th>20</th> </tbody> <tbody></tbody>

med  62

low  27

high  99

low   7

low  47
```

```
In [168... # visualization using matplotlib
```

```
In [169... df_visual = df.copy()
```

```
In [170... df_visual.columns
```

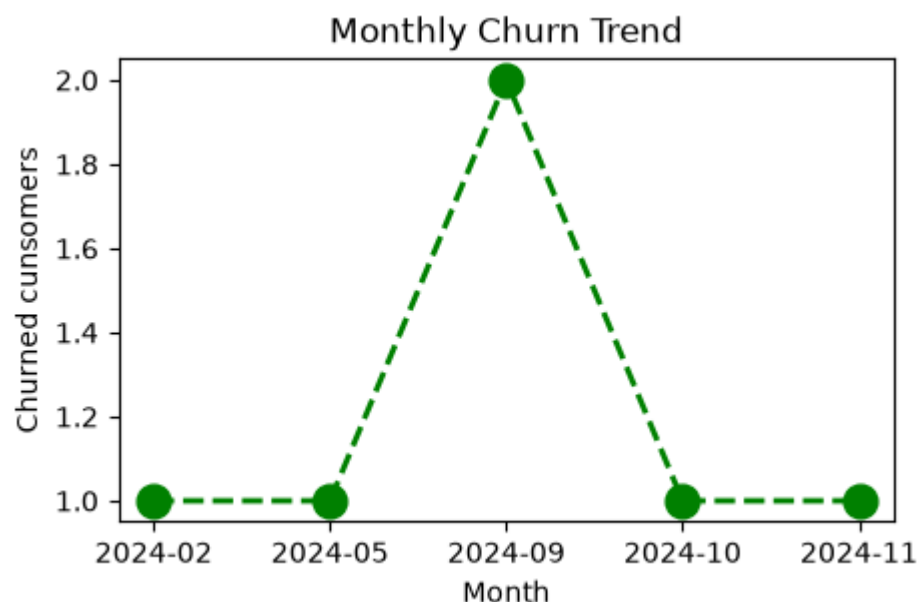
```
Out[170... Index(['customerid', 'subscription_start_date', 'subscription_type',
        'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
        'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
        'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
        'complaint_date', 'escalations', 'csat_score', 'complaint_count',
        'tenure_days', 'churn_risk'],
        dtype='str')
```

```
In [171... # 4.1 Monthly Churn Trend (Time Series KPI)
df_visual['cancellation_month'] = df_visual['cancellation_date'].dt.to_period('M')

churn_trend = df_visual[df_visual['churn_flag'] == 1].groupby('cancellation_month').size()

plt.figure(figsize=(5,3))
plt.plot(churn_trend.index.astype(str),churn_trend.values, color='green', marker='o', linestyle

plt.title('Monthly Churn Trend')
plt.xlabel('Month')
plt.ylabel('Churned cunsomers')
plt.show()
```

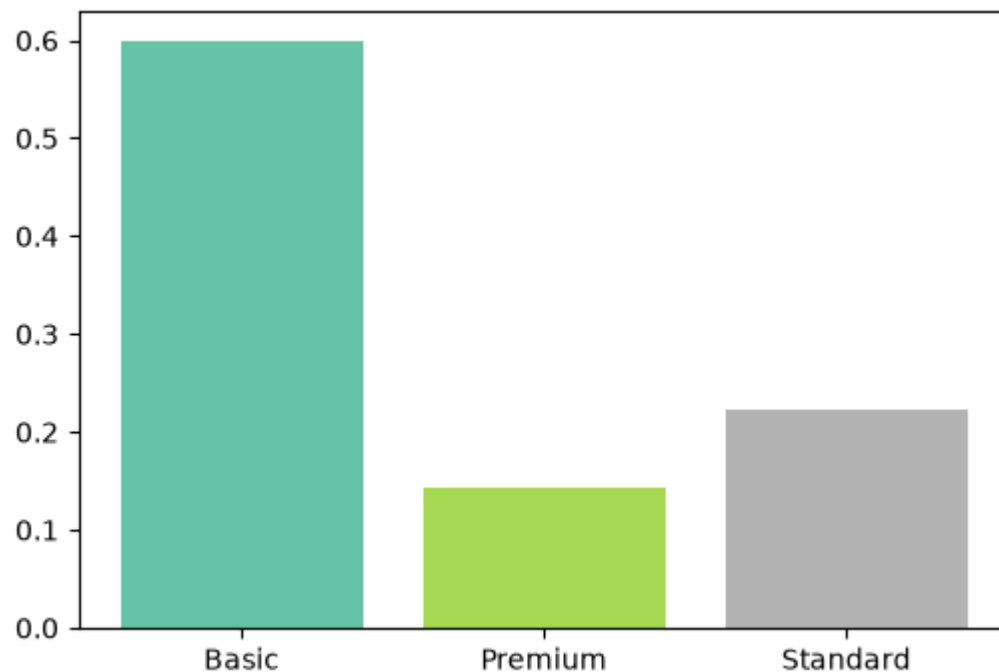


In [172...

```
# 4.2 churn by plan type
churn_plan = df_visual.groupby('plan_type')['churn_flag'].mean()

#colors = ['yellow', 'orange', 'red']
colors = plt.cm.Set2(np.linspace(0, 1, len(churn_plan)))
plt.figure(figsize=(6,4))

plt.bar(churn_plan.index, churn_plan.values, color = colors)
plt.show()
```

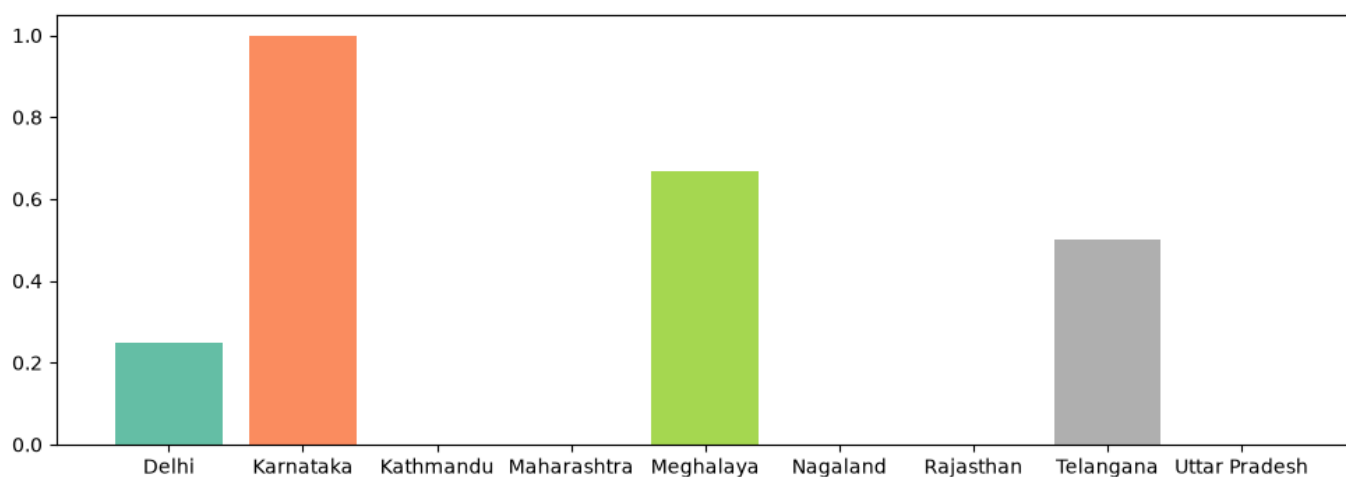


In [173...

```
# 4.2 churn by state
churn_plan = df_visual.groupby('state')['churn_flag'].mean()

#colors = ['yellow', 'orange', 'red']
colors = plt.cm.Set2(np.linspace(0, 1, len(churn_plan)))
plt.figure(figsize=(12,4))

plt.bar(churn_plan.index, churn_plan.values, color = colors)
plt.show()
```



In []:

In [174...

```
# visulization using seaborn
```

```
In [175... # encoding - convert str to numeric so thst we can find corr between features
df_visual.columns
```

```
Out[175... Index(['customerid', 'subscription_start_date', 'subscription_type',
      'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
      'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
      'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
      'complaint_date', 'escalations', 'csat_score', 'complaint_count',
      'tenure_days', 'churn_risk', 'cancellation_month'],
      dtype='str')
```

```
In [176... import warnings
warnings.filterwarnings("ignore")
```

```
In [177... df_encoded = df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag', 'churn_risk', 'cancellation_month']]

categorical_cols = ['plan_type', 'contract_type', 'churn_risk']

for col in categorical_cols:
    df_encoded[col] = df_encoded[col].astype('category').cat.codes
```

```
In [178... df_encoded.head()
```

```
Out[178... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th {
vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th>
<th>plan_type</th> <th>contract_type</th> <th>churn_score</th> <th>churn_flag</th>
<th>churn_risk</th> <th>escalations</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th>
<th>3</th> <th>4</th> </tbody> <tbody></tbody>
```

```
2  0  12  0  1  NaN
```

```
1  0  91  1  0    Y
```

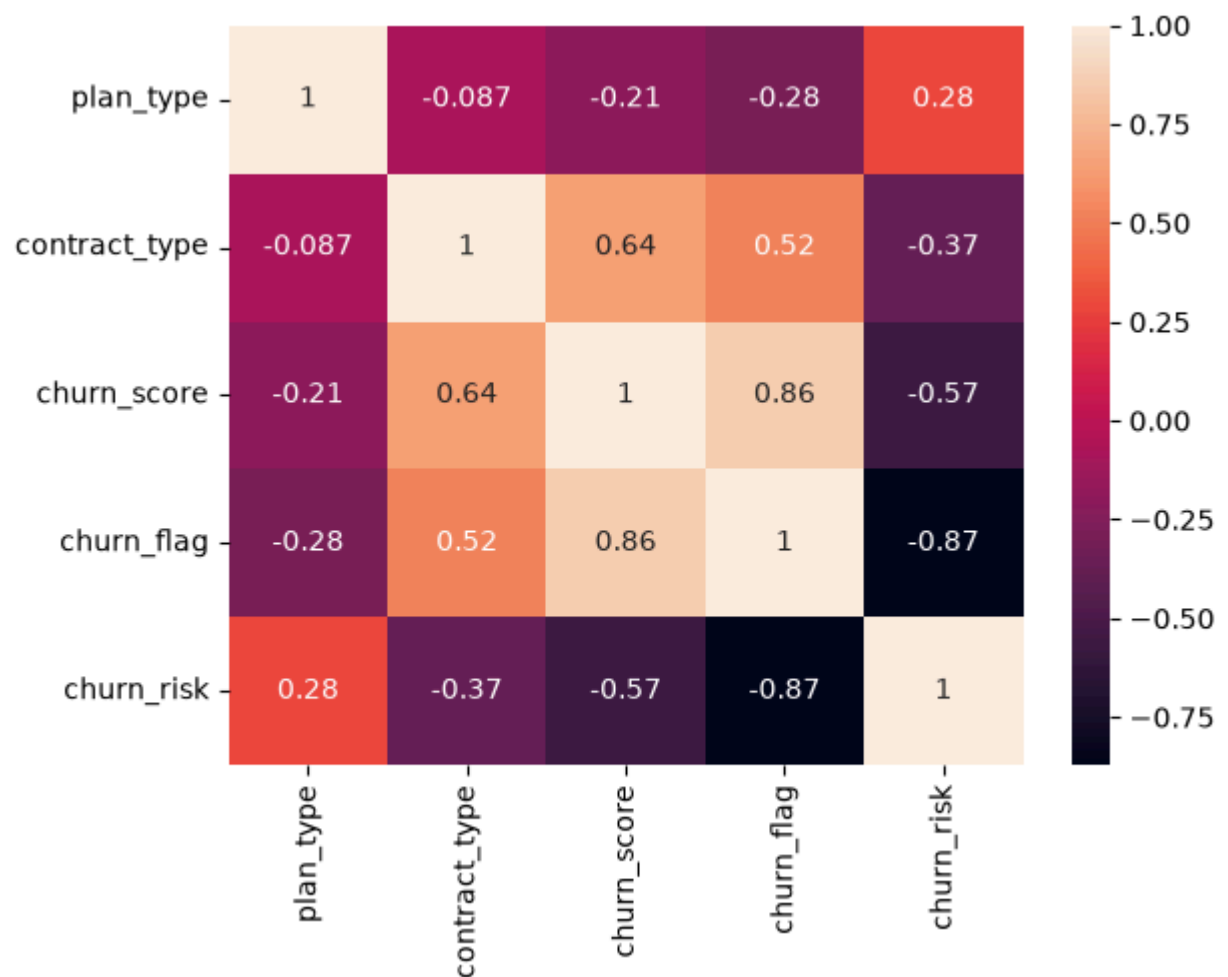
```
0  1  34  0  1  NaN
```

```
1  0   8  0  1  NaN
```

```
2  1  88  1  0    Y
```

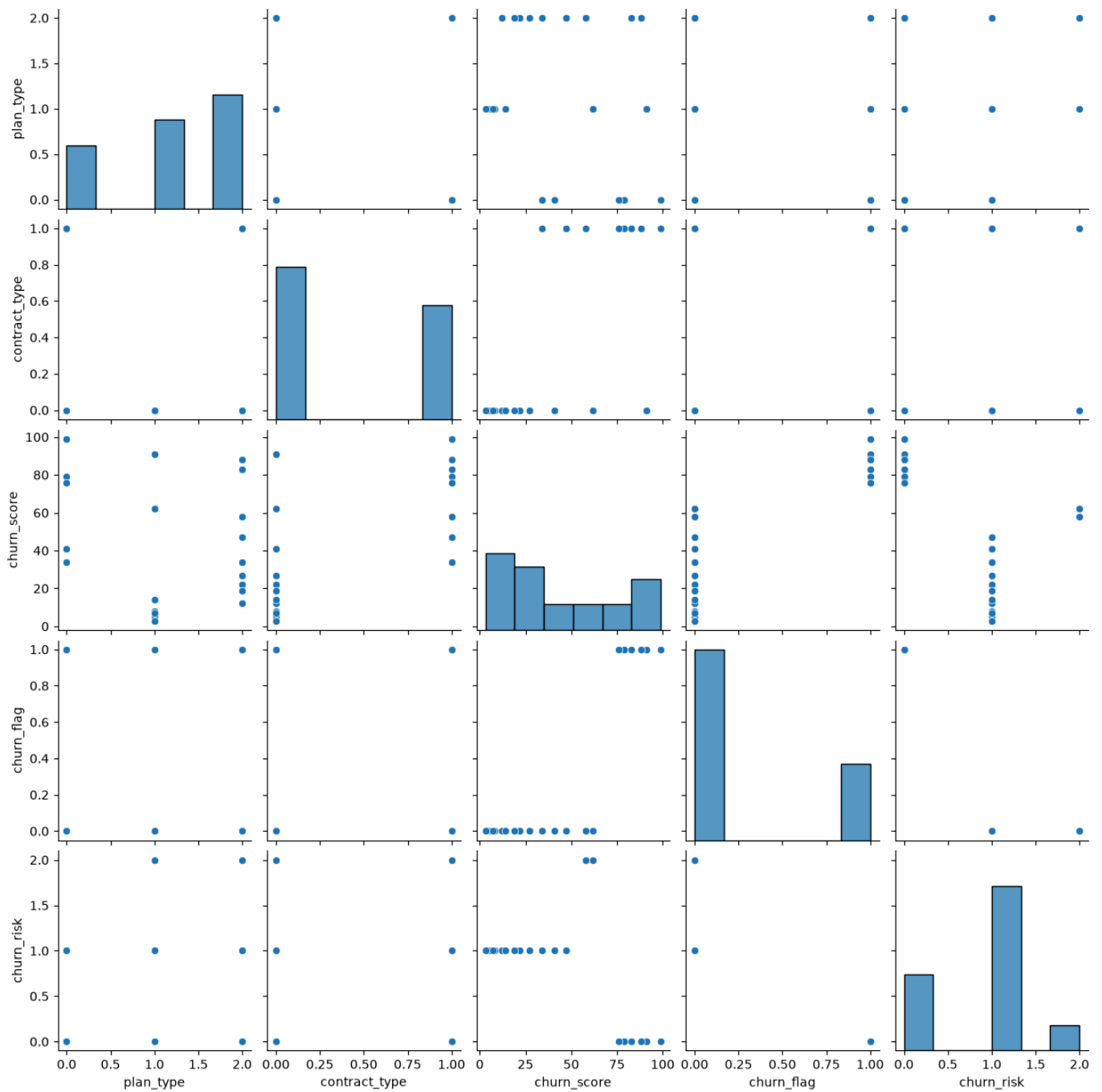
```
In [179... # Heatmap (correlation matrix)
sns.heatmap(df_encoded.corr(numeric_only=True), annot=True)
```

```
Out[179... <Axes: >
```



```
In [180... # pairplot - relationship in dataset  
sns.pairplot(df_encoded)
```

```
Out[180... <seaborn.axisgrid.PairGrid at 0x22aac37f770>
```

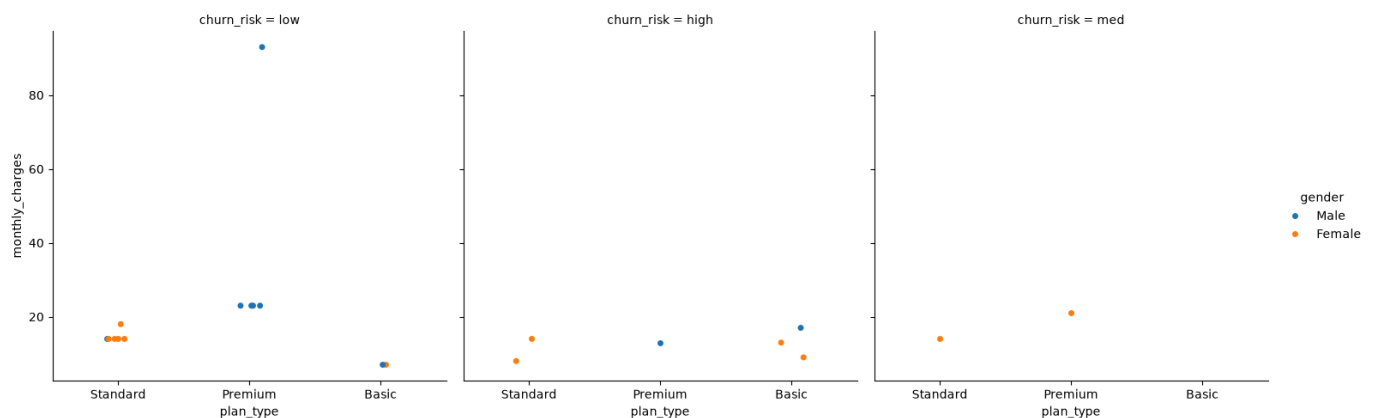


In [182...

```
# catplot/facegrid plot
sns.catplot(data=df_visual,
            x='plan_type',
            y='monthly_charges',
            hue='gender',
            col='churn_risk')
```

Out[182...

<seaborn.axisgrid.FacetGrid at 0x22aac37f4d0>



In []:

In [221... *# pivot Table*

```
In [222... pd.pivot_table(  
    df_visual,  
    values='churn_flag',  
    index='plan_type',  
    aggfunc='mean'  
)
```

Out[222... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th {
vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th>
<th>churn_flag</th> <th>plan_type</th> <th></th> </thead> <tbody> <th>Basic</th>
<th>Premium</th> <th>Standard</th> </tbody> <tbody></tbody>

0.600000
0.142857
0.222222

In [223... df_visual.columns

Out[223... Index(['customerid', 'subscription_start_date', 'subscription_type',
'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
'complaint_date', 'escalations', 'csat_score', 'complaint_count',
'tenure_days', 'churn_risk', 'cancellation_month'],
dtype='str')

```
In [224... pd.pivot_table(  
    df_visual,  
    index='plan_type',  
    values=['monthly_charges', 'customerid', 'churn_flag'],  
    aggfunc={  
        'monthly_charges': 'sum',  
        'customerid': 'nunique',  
        'churn_flag': 'mean'}  
)
```

Out[224... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th {
vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th>
<th>churn_flag</th> <th>customerid</th> <th>monthly_charges</th> <th>plan_type</th> <th>
</th> <th></th> <th></th> </thead> <tbody> <th>Basic</th> <th>Premium</th>
<th>Standard</th> </tbody> <tbody></tbody>

0.600000 5 52.95
0.142857 7 218.93
0.222222 9 123.91

In []:

In [225... *# working with SQL in python with (pandas)*

In [226...

```
# insert data

cursor = conn.cursor()

cursor.execute(
    """
    INSERT INTO users VALUES
    ('Mahadev', 'India', 5000),
    ('Rishabh', 'Germany', 2500),
    ('Vishaka', 'India', 3500)
    """
)

#commit and save
conn.commit()

print("Data Inserted Successfully!")
```

Data Inserted Successfully!

In [227...

```
# check inserted data in table

conn = sqlite3.connect('test_database.sqlite')
query = """SELECT * FROM users"""

df_result = pd.read_sql(query, conn)

df_result.head()
```

Out[227...

```
<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th {
vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th>
<th>first_name</th> <th>country</th> <th>budget</th> </thead> <tbody> <th>0</th>
<th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>
```

Mahadev	india	5000
---------	-------	------

Rishabh	Germany	2500
---------	---------	------

Vishaka	india	3500
---------	-------	------

Mahadev	India	5000
---------	-------	------

Rishabh	Germany	2500
---------	---------	------

In [228...

```
# aggregation

query = """
    SELECT country, sum(budget)as total_budget
    FROM users
    GROUP BY country
    """

df_agg = pd.read_sql(query, conn)
df_agg
```

Out[228...

```
<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th {
vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th>
<th>country</th> <th>total_budget</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th>
</tbody> <tbody></tbody>
```

Germany	12500
---------	-------

India	34000
-------	-------

india	8500
-------	------

In [212...

```
conn.close()
```

In []: